Rust Verification Through Lean 4 Tooling Investigation report

Discovery report for Milestone 2 — Implementation and Verification Ethereum

Delivered: March 02, 2026

runtime verification

Table of Contents

- Introduction
- Our Contributions
  - 1. Upstream Contributions to Hax
  - 2. Pipeline Example: ADC Verification (Annotation-Based)
  - 3. Pipeline Example: FRI Folding Verification (ArkLib Integration)
  - 4. Hax Lean Backend Gap Analysis
  - 5. Stress-testing Hax with Prime Field Arithmetic
    - Current Extraction Status
- Conclusion and Future Work

Introduction

This report summarises the work completed during Phase 2 of the project, whose objective was to build a functional verification workflow and produce initial formal proofs for extracted cryptographic Rust code.

The efforts and achievements of this phase can be summarized as follows:

- Established two end-to-end pipelines that take Rust code through Hax extraction into Lean 4 and produced machine-checked proofs.
- Verified key correctness properties of the Plonky3 FRI folding kernel.
- Continued on testing Hax's capabilities for more complex rust models for field arithmetic. Particularly adding the Montgomery 31-bit and Koalabear prime fields.
- As a result of the above, contributed issues and upstream improvements to the Hax toolchain and documented gaps in the Lean backend that inform future work.

Our Contributions

1. Upstream Contributions to Hax

We merged **5 pull requests** into the Hax repository:

PR	Title	Description
#1927	*lean: keep Rust crate/module names unchanged*	Fixed the Lean backend to preserve original Rust crate/module naming instead of capitalising the first letter, which caused Lake configuration friction.
#1932	*fix(lean): Extract correct `PhantomData` structure*	Fixed the way `impl` ementations of `struct` s where extracted if these had a `PhantomData` item.
#1950	*Remove BinOp resugaring*	Preparatory work for a follow up PR (#1963) that closed an old-ish issue (Rethink the `BinOps` resugaring) about Hax's internal representation of certain operations.
#1959	*ADC example, documented*	Added a fully documented addition-with-carry (ADC) proof example demonstrating the `hax_mvcgen` , `specset "bv"` , and `hax_bv_decide` workflows.
#1963	*Add more binops*	Added the logic needed for the correct generation of the `not` operator and others, and prettified the generation of these and the existing operators for boolean and integer types.

We also raised **12 issues** in the Hax repository, of which **9 have been closed as resolved** (including our own fix in PR #1927).

After discussing with the maintainers, some issues were closed as either duplicates or too complex to address:

Issue	Status
#1911 [Lean] missing Core_models.Ops.Arith.Add instance for +? after cast_op (type inference)	Closed (fixed in #1925)
#1912 [Lean] Add instance missing for expression involving shift-left (<<< ?) with i32 shift amount	Closed (fixed in #1925)
#1913 [Lean] Extracted function returns Rust_primitives.Hax.Tuple2, but .fst/.snd projections are missing	Closed
#1914 [Lean]cargo hax into lean generates a capitalized module/file name (e.g. Hax_adc_poc.lean) causing Lake configuration friction	Closed (fixed in #1927)
#1915 [Lean] hax_bv_decide fails on extracted goals: “None of the hypotheses are in the supported BitVec fragment after applying preprocessing.”	Closed
#1916 [Lean] Extraction emits monadic binds for pure arithmetic, so type inference and automation gets harder	Closed
#1928 [Lean] Spurious `bind` when using other implementation's constants	Closed (fixed in #1929)
#1930 [Lean] Incorrect extraction of `PhantomData`	Closed (fixed in #1932)
#1939 [Lean] No `Playground.lean` file in Hax Playground	Closed
#1940 [Lean] Missing anonymous bind when operating with `const` parameters	Closed (fixed in #1941)
#1948 [Lean] Unimplemented `not` operator	Closed (fixed in #1963)
#1964 [Lean] Incorrect placement of `RustM.of_isOk` in generated trait items	Open (waiting on closing [Lean] Implemented items that use other associated items produce invalid Lean before addressing this one)

2. Pipeline Example: ADC Verification (Annotation-Based)

Repository: runtimeverification/hax-annotations-adc-verification

This pipeline demonstrates formal verification of a 32-bit addition-with-carry (ADC) function by annotating Rust source code directly with Hax specifications. The verified properties include:

- No panics occur during execution.
- Carry values are constrained to {0, 1}.
- The arithmetic identity `a + b + carry = sum + 2^32 * carry_out` holds in 64-bit arithmetic.

This example also served as the primary vehicle for discovering and reporting the 6 Hax issues listed above.

3. Pipeline Example: FRI Folding Verification (ArkLib Integration)

Repository: runtimeverification/p3-hax-lean-fri-pipeline

This pipeline extracts Plonky3's `compute_log_arity_for_round` function (a single FRI folding step) from Rust into Lean 4 via Hax, and connects the extracted code to ArkLib's formal FRI specifications.

Proved theorems (in `FoldingCorrectness.lean`):

- **`arity_respects_target_distance`** — The computed log-arity never exceeds the distance between the current domain height and the final height (`result ≤ log_current_height - log_final_height`).
  This ensures that a single folding round never overshoots the target domain size.
- **`arity_respects_max_bound`** — The computed log-arity is always bounded by `max_log_arity` (`result ≤ max_log_arity`), for both the `None` and `Some` branches of the next-input parameter. This holds unconditionally for any successful call.

Both proofs operate directly on the Hax-extracted monadic code (`RustM`), reasoning through checked arithmetic (`-?`), constructor case-splits on `Core_models.Option`, and boolean conditionals from `Machine_int.lt`.

4. Hax Lean Backend Gap Analysis

Through the proof work, we identified concrete gaps in Hax's Lean backend that make formal verification harder than necessary. A detailed report with suggested fixes has been compiled in `HAX_MISSING_LEMMAS.md`. The key gaps are:

Gap	Severity
No `@[simp]` discriminator lemmas for `RustM` constructors	High
No characterisation lemmas for checked arithmetic (`-?` , `+?` , `*?`)	High
Bool `if` -then-else vs. `ite` mismatch in generated code	High
No bridge lemma from `Machine_int.lt` to `Prop`	Medium
No `omega` / `norm_cast` integration for `USize64`	Medium
`Core_models.Option` constructor order reversed vs. stdlib	Low

5. Stress-testing Hax with Prime Field Arithmetic

Continuing our work on extracting a model of Plonky3's field arithmetic yielded a number of issues and contributions to the Hax tool. In particular, we focused this time in creating a model for the KoalaBear prime field, which is built using a model for a generic Montgomery 31-bit prime field.
A Montgomery 31-bit prime field is a prime field whose elements are represented in Montgomery form and its primer order fits within 31 bits. Plonky3 has two such fields, KoalaBear and BabyBear.

For more details on the model and its extraction into Lean 4 with Hax, see the corresponding pull request #1.

Current Extraction Status

The entirety of the effort on this side of the project was spent on the following tasks:

- Understanding the core logic and architecture of our targeted Rust that we model after.
- Generating the models themselves.
- Tweaking the models to adapt as best as possible to Hax's idiosyncrasies.
- Repdocucing encountered problems in PoCs to file issues to Hax's repository.
- To the best of our capacity, submit fixes to encountered problems.

The amount of effort dedicated into producing the generated Lean and the current state of it have preclude us from attempting any theorem proving or linkage with the ArkLib and CompPoly specificaiton libraries.

Conclusion and Future Work

Phase 2 established two working Rust-to-Lean verification pipelines and produced the first machine-checked proofs of FRI folding correctness properties against ArkLib specifications.
We also defined a self contained and somewhat problematic field arithmetic Rust model for Hax, valuable for insight-gathering and further testing.
The upstream contributions (5 merged PRs, 12 issues) directly improved the Hax toolchain for the broader community, and the gap analysis provides a concrete roadmap for making Hax's Lean backend more proof-friendly.

In the next phase, we plan to build a similar proof-of-concept using Aeneas as an alternative Rust-to-Lean extraction tool. This will allow a direct comparison of the two approaches — Hax (annotation-driven, monadic extraction) vs. Aeneas (MIR-based, pure functional extraction) — in terms of translation coverage, proof complexity, and tactic automation support. The goal is to determine which pipeline is best suited for verifying real-world cryptographic code at scale, and to gauge Aeneas against the more problematic Rust examples we have attempted with Hax.